```c
#include <stdio.h>

#define MAX 10
#define INF 999

int main() {
    int n, cost[MAX][MAX], dist[MAX][MAX];
    int nextHop[MAX][MAX], i, j, k, changed;
    int src, dest, path[MAX], count;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter cost matrix (999 for no direct link):\n");

    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            dist[i][j] = cost[i][j];

            if (i == j)
                nextHop[i][j] = i;
            else if (cost[i][j] != INF)
                nextHop[i][j] = j;
            else
                nextHop[i][j] = -1;
        }
    }

    printf("\nInitial Routing Tables:\n");
```

```
for (i = 0; i < n; i++) {
    printf("\nRouter %c:\n", 'A' + i);
    printf("Destination\tCost\tNext Hop\n");

    for (j = 0; j < n; j++) {
        printf("%c\t\t", 'A' + j);

        if (dist[i][j] == INF)
            printf("INF\t");
        else
            printf("%d\t", dist[i][j]);

        if (nextHop[i][j] == -1)
            printf("-\n");
        else
            printf("%c\n", 'A' + nextHop[i][j]);
    }
}

do {
    changed = 0;
    int newDist[MAX][MAX];
    int newNext[MAX][MAX];

    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            newDist[i][j] = dist[i][j];
            newNext[i][j] = nextHop[i][j];
        }
    }
```

```
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            if (i == j)
                continue;

            for (k = 0; k < n; k++) {
                if (cost[i][k] != INF &&
                    dist[k][j] != INF &&
                    cost[i][k] + dist[k][j] < newDist[i][j]) {

                    newDist[i][j] =
                        cost[i][k] + dist[k][j];

                    newNext[i][j] = k;
                    changed = 1;
                }
            }
        }
    }

    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            dist[i][j] = newDist[i][j];
            nextHop[i][j] = newNext[i][j];
        }
    }

} while (changed);

printf("\nFinal Routing Tables:\n");
```

```
for (i = 0; i < n; i++) {
    printf("\nRouter %c:\n", 'A' + i);
    printf("Destination\tCost\tNext Hop\n");

    for (j = 0; j < n; j++) {
        printf("%c\t\t", 'A' + j);

        if (dist[i][j] == INF)
            printf("INF\t");
        else
            printf("%d\t", dist[i][j]);

        if (nextHop[i][j] == -1)
            printf("-\n");
        else
            printf("%c\n", 'A' + nextHop[i][j]);
    }
}

printf("\nEnter source node (0 to %d): ", n - 1);
scanf("%d", &src);

printf("Enter destination node (0 to %d): ", n - 1);
scanf("%d", &dest);

if (src < 0 || src >= n || dest < 0 || dest >= n) {
    printf("Invalid node number.\n");
    return 1;
}

if (dist[src][dest] == INF) {
```

```
    printf("Destination is unreachable.\n");
    return 0;
}

printf("\nShortest path: ");
printf("%c", 'A' + src);

count = 0;
path[count++] = src;
i = src;

while (i != dest && count < n) {
    i = nextHop[i][dest];

    if (i == -1)
        break;

    path[count++] = i;
}

if (i != dest) {
    printf("\nUnable to reconstruct path.\n");
    return 1;
}

for (i = 1; i < count; i++)
    printf(" -> %c", 'A' + path[i]);

printf("\nMinimum cost: %d\n", dist[src][dest]);

return 0;
```

}

```
Enter number of nodes: 4
Enter cost matrix (999 for no direct link):
0 1 17 999
1 0 2 8
7 2 8 1
999 8 1 0

Initial Routing Tables:

Router A:
Destination Cost    Next Hop
A       0   A
B       1   B
C       17  C
D       INF -

Router B:
Destination Cost    Next Hop
A       1   A
B       0   B
C       2   C
D       8   D
```

```
Router C:
Destination Cost    Next Hop
A       7   A
B       2   B
C       8   C
D       1   D

Router D:
Destination Cost    Next Hop
A       INF -
B       8   B
C       1   C
D       0   D

Final Routing Tables:

Router A:
Destination Cost    Next Hop
A       0   A
B       1   B
C       3   B
D       4   B

Router B:
Destination Cost    Next Hop
A       1   A
B       0   B
C       2   C
D       3   C

Router C:
Destination Cost    Next Hop
A       3   B
B       2   B
C       8   C
D       1   D

Router D:
Destination Cost    Next Hop
A       4   C
B       3   C
C       1   C
D       0   D

Enter source node (0 to 3):
```